

SC102 L10: 骨骼动画与 IK ——详细学习笔记

涵盖：骨骼动画 | 正向动力学 (FK) | 2D 骨骼绑定 | 逆向动力学 (IK) |
Limb/CCD/FABRIK Solver

1 骨骼动画基础

[p.4-5]

骨骼动画 (Skeletal Animation / Rigging) 是由“骨骼”控制物体外形的技术。骨骼是一组相互连接的层级结构——一根骨骼由一个方向和一个长度组成。骨骼 A 的头部与骨骼 B 的尾部相连，则 A 是 B 的“孩子”（A 的位置和旋转继承自 B）。

2D 动画的两种类型：

- **帧动画 (Frame Animation)：**每 k 帧替换一张图片。优点：每一帧完全可控；缺点：占用空间大、帧率有限（关键帧间无法插值）。
- **骨骼动画：**程序化动画。优点：实时性（帧率由游戏帧率决定）、占用空间小（只记录关键帧骨骼数据）、可在运行时修改控制。

3D 骨骼动画：在 3D 开发中，骨骼动画间接控制物体形态（通过骨骼权重），比直接修改 Transform 更适合制作人物/动物等自然运动的动画。绝大部分 3D 游戏都用骨骼动画制作人物动作。

2 正向动力学 (Forward Kinematics, FK)

[p.7-9]

FK = 已知过程推导结果：已知各关节角度，计算出末端位置。

FK 系统关键概念：

- **Joint (关节)：**骨骼头部，控制旋转
- **Links (连接)：**骨骼尾部，控制长度和骨骼间连接
- **End-Effector (末端器)：**整条骨骼链的尾部
- **Kinematic Chain (动力学链)：**一整条相连的骨骼

FK 末端器状态计算：

```
// b1的头部 = b0的尾部 = b0头部 + b0方向 * b0长度
Vector3 arm0End = arm0.position + arm0.right * armLength;
arm1.position = arm0End;
// b1的旋转 = b1自身旋转 + b0的旋转（继承）
arm1.rotation = Quaternion.Euler(
    Vector3.forward * arm1Rotation + arm0.rotation.eulerAngles);
```

3 2D 骨骼动画实践 (FK)

[p.10-12]

骨骼绑定流程 (Skinning Editor)：

1. 导入 PSB 文件 → 打开 Sprite Editor→Skinning Editor
2. Auto Geometry：自动多边形细分
3. Create Bones：创建骨骼
4. Auto Weights：自动生成骨骼权重（“蒙皮”= 建立骨骼与外形联系）
5. Preview Pose：预览绑骨效果
6. 拖入场景 → Animation 窗口录制关键帧动画

Skinning Editor 四大面板：Pose（姿态总览）、Bones（骨骼）、Geometry（几何细分）、Weights（权重——每部分受各骨骼影响的权重）。

4 逆向动力学 (Inverse Kinematics, IK)

[p.14-15]

IK = 通过结果逆向推导过程：已知末端要到达的位置 (Target)，计算出各关节应有的角度。

核心概念：

- **Target：**末端器应到达的目标位置
- IK 系统实时计算每根骨骼的旋转，让 End Effector 尽可能接近 Target
- IK 可能有多个解——需要找“最优解”

数值解 vs 解析解：

- **解析解 (Analytical)：**用公式表达的完全精确解
- **数值解 (Numerical)：**近似数值。游戏开发中数值解更重要——可以直接使用达到 IK 效果

三种常见 2D IK 数值解法:

1. **Limb Solver**: 最简单, 只支持 2 根骨骼
2. **CCD Solver**: 迭代逼近, 支持任意长度
3. **FABRIK Solver**: 更成熟, 前后双向计算

4.1 Limb Solver

[p.16]

算法:

1. 计算根节点到 Target 的距离 d
2. 若 $d >$ 两骨骼总长 s : 无法到达 $\rightarrow$ 将两骨骼向 Target 排成一线
3. 若 $d \leq s$: 可以到达 $\rightarrow$ 用余弦定理计算两骨骼间夹角和根骨骼偏转角

限制: 两根骨骼夹角只能在 0-180 度 (只能朝一个方向弯曲); 只支持长度为 2 的 IK; 不够灵活 (无法限制旋转范围)。

4.2 CCD Solver

[p.17]

算法 (迭代逼近):

1. 从 End Effector 开始
2. 从末端向根骨骼遍历, 计算每根骨骼到 End Effector 和到 Target 的夹角, 旋转骨骼最小化此夹角
3. 检查 End Effector 和 Target 距离是否够近 $\rightarrow$ 是则停止
4. 未通过则重新迭代, 最多 x 次

特点: 灵活通用、计算量中等。缺点: 不总能找到“看起来最自然”的解。

4.3 FABRIK Solver

[p.18]

算法 (前后双向计算):

1. 从根骨骼开始
2. **前向 (Forward)**: 从根骨骼 $\rightarrow$ End Effector, 每根骨骼放在上一骨骼和 Target 连线方向上。结束后 End Effector 一定在 Target 上

3. 反向 (Backward): 从 End Effector → 根骨骼, 同上。结束后所有骨骼”拉回”根骨骼, End Effector 仍在 Target 上
4. 迭代直到收敛或达到最大次数

特点: 成熟方法, 找到的解法较自然, 迭代次数通常少于 CCD。

4.4 三种 IK 算法对比

[p.19]

算法	适用	效果	性能
Limb	2 骨骼	自然	低
CCD	多骨骼	不一定自然	较高
FABRIK	多骨骼	自然	较高

5 Unity 2D IK 实践

[p.20]

使用 Unity 的 IKManager2D 组件制作 IK 动画:

1. 绑定好骨骼的 prefab 拖入场景
2. 创建空物体, 添加 IKManager2D 组件
3. IKSolvers 下添加 IKSolver, 选择算法 (Limb/CCD/FABRIK)
4. 拖入 Effector 和 Target 的 Transform
5. 在 Animation 窗口对 Target 录制 Transform 动画

6 课后习题详解

[p.21]

6.1 题 1: CCD 和 FABRIK 伪代码

CCD 伪代码:

```
CCD(bones, target, maxIterations):
    for iter in 1..maxIterations:
        for bone in bones (from end to root):
            v1 = endEffector - bone.position
            v2 = target - bone.position
            angle = AngleBetween(v1, v2)
```

```

    RotateBoneBy(bone, angle)
    if Distance(endEffector, target) < threshold:
        break

```

FABRIK 伪代码:

```

FABRIK(bones, target, maxIterations):
    for iter in 1..maxIterations:
        // Forward: root to end
        bones[last].position = target
        for i from n-1 down to 0:
            dir = (bones[i+1].pos - bones[i].pos).normalized
            bones[i+1].pos = bones[i].pos + dir * boneLength[i]
        // Backward: end to root
        bones[0].position = root
        for i from 0 to n-1:
            dir = (bones[i+1].pos - bones[i].pos).normalized
            bones[i+1].pos = bones[i].pos + dir * boneLength[i]
        if Distance(endEffector, target) < threshold:
            break

```

6.2 题 2: 游戏中 IK 的 5 个常见功能

1. 脚部贴地 (**Foot IK**): 角色站在斜坡/台阶上, 脚部自动适应地面高度
2. 手部抓取 (**Hand IK**): 角色伸手抓取物体, 手自动到达物体位置
3. 头部注视 (**Look At IK**): 角色头部/眼球跟随目标物体
4. 尾巴/触手动画: 尾巴末端自动跟随某目标点
5. 多足生物行走: 蜘蛛等生物脚根据地面自适应摆放

6.3 题 3: 火柴人待机与走路骨骼动画

使用 Sprite Editor 绑骨后:

- 待机动画: 略微上下浮动身体 (调整 Root 骨骼 Y 位置)、手臂轻微摆动、循环
- 走路动画: 手脚交替前后摆动 (类似正弦曲线 offset)、身体随步伐上下起伏、关键帧循环